

SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING

COURSE CODE: CSA43

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. (L2)

Assessment Tool: Data Interpretation (Medium)

Weightage: 20%

1. HTTP Response Analysis

1. Identify any incorrect or invalid fields in the header

Given Header:

HTTP/1.1 200 OK

Date: Mon, 01 Apr 2026 10:00:00 GMT

Content-Type: text/html

Content-Length: -150

Connection: keep-alive

Issues:

- Content-Length: -150 is **invalid** because Content-Length cannot be negative.
- The Date, Content-Type, and Connection headers are valid.
- The status line HTTP/1.1 200 OK is correctly formatted.

2. Analyze the issue with Content-Length and explain its impact

- Content-Length specifies the size of the response body in **bytes**.
- A negative value is not allowed by the HTTP specification.
- The browser cannot determine the correct response size.
- This may cause:
 - Failure to load the page.
 - Connection errors.
 - Browser ignoring the response or terminating the connection.

3. Determine whether the status code matches the response correctness

- **200 OK** indicates that the request was processed successfully.
- Since the response contains an invalid Content-Length, the response is **not completely correct**.
- A 200 status code should only be returned with a valid response.

4. Suggest corrections for the identified errors

Correct Header:

HTTP/1.1 200 OK

Date: Mon, 01 Apr 2026 10:00:00 GMT

Content-Type: text/html

Content-Length: 150

Connection: keep-alive

Improvements:

- Use a valid positive Content-Length.
- Ensure the Content-Length matches the actual response body size.
- Validate HTTP headers before sending the response.

2. DOM Structure Analysis

1. Identify missing or improperly closed elements

Original Code:

```
<html>
<head>
<title>TestPage</title>
</head>
<body>
<div>
<p>HelloWorld
</div>
</body>
</html>
```

Issues:

- <p> tag is not closed.
- <div> is closed before the paragraph is explicitly closed.
- Missing <!DOCTYPE html>.
- Missing lang attribute in <html>.

2. Analyze how the browser will interpret this structure

Modern browsers automatically correct HTML errors.

The browser will interpret it as:

```
<div>
  <p>HelloWorld</p>
</div>
```

This behavior is called **error recovery**.

3. Determine the impact on rendering and layout

Possible impacts:

- Most modern browsers display the page correctly.
- Different browsers may repair the HTML differently.
- Complex pages may have unexpected layouts.
- Invalid HTML can reduce accessibility and SEO quality.

4. Suggest corrections to make the DOM valid

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Test Page</title>
</head>
<body>
<div>
  <p>Hello World</p>
</div>
</body>
</html>
```

3. GET vs POST in Login System

1. Interpret the differences in data transmission between GET and POST

GET	POST
Sends data in the URL	Sends data in the request body
Limited data size	Supports large data
Can be bookmarked	Cannot be bookmarked
Used for retrieving data	Used for submitting data
Less secure	More secure than GET

2. Analyze which method is more suitable for login and why

POST is the preferred method because:

- Credentials are sent in the request body instead of the URL.
- Passwords are not exposed in browser history.
- Sensitive information is less likely to appear in server logs.
- Better suited for authentication operations.

3. Identify security risks in using GET for authentication

Using GET for login may:

- Expose usernames and passwords in the URL.
- Store credentials in browser history.
- Leak credentials through bookmarks.
- Record credentials in server logs.
- Expose credentials via the HTTP Referer header.

Example:

/login?user=admin&pass=1234

4. Suggest best practices for secure login implementation

- Use the **POST** method.
- Always use **HTTPS**.
- Hash and salt passwords on the server.
- Implement CSRF protection.
- Validate inputs on both client and server.
- Use secure session management.
- Enable multi-factor authentication (MFA) where appropriate.
- Never store passwords in plain text.

4. HTML Code Inspection

1. Identify missing closing tags

Original Code:

<!DOCTYPE html>

```
<html>
<head>
<title>Sample</title>
</head>
<body>
<h1>Welcome
<p>This is a paragraph

</body>
</html>
```

Missing tags:

- </h1>
- </p>

2. Analyze issues with the tag

The tag is valid because it is a **void element** and does not require a closing tag.

However, it should include:

- alt attribute for accessibility.
- Optional width and height attributes to reduce layout shifts.

Example:

```

```

3. Determine how browsers will handle these errors

Browsers automatically:

- Close the <h1> tag when encountering the <p> tag.
- Close the <p> tag before the end of the <body>.
- Display the image if it exists.
- Show a broken image icon if the image cannot be found.

4. Suggest a corrected version of the code

```
<!DOCTYPE html>
<html lang="en">
```

```
<head>
  <meta charset="UTF-8">
  <title>Sample</title>
</head>
<body>
<h1>Welcome</h1>
<p>This is a paragraph.</p>

</body>
</html>
```

5. Browser-Server Communication Flow

1. Interpret each step in the communication flow

- 1. User enters a URL**
The user types a website address (URL) into the browser.
- 2. DNS lookup occurs**
The browser contacts a DNS server to translate the domain name into an IP address.
- 3. HTTP request is sent**
The browser sends an HTTP or HTTPS request to the web server.
- 4. Server processes the request**
The server retrieves or generates the requested content and prepares a response.
- 5. Response is returned**
The server sends an HTTP response containing status codes, headers, and content.
- 6. Browser renders the page**
The browser parses HTML, CSS, and JavaScript, builds the DOM and CSSOM, and displays the webpage.

2. Identify where delays can occur

Delays may occur during:

- DNS lookup.
- Internet/network latency.
- Server processing time.
- Database queries.
- Downloading large files (images, videos, scripts).
- Browser rendering and JavaScript execution.

3. Analyze the role of DNS in the process

DNS (Domain Name System):

- Converts human-readable domain names into IP addresses.
- Enables the browser to locate the correct web server.
- Acts like the "phonebook" of the Internet.
- Without DNS resolution, the browser cannot connect to the server using the domain name.

4. Suggest optimizations to improve performance

- Enable DNS caching.
- Use a Content Delivery Network (CDN).
- Compress resources using Gzip or Brotli.
- Minify HTML, CSS, and JavaScript files.
- Optimize and compress images.
- Use browser caching.
- Reduce server response time.
- Load JavaScript asynchronously or defer its execution.
- Use HTTP/2 or HTTP/3 for faster communication.

Code of Conduct:

A.ADARSH (192365002) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: